

Department of Statistics and Data Science

MDS482-4 PROJECT



CHRIST
UNIVERSITY
B E N G A L U R U , I N D I A

Declared as Deemed to be University under Section 3 of UGC Act 1956

SYSTEM DESIGN REPORT

PAWMAP

A Community-Driven Stray Animal Rescue and Welfare Platform

Prepared by

Anavadhya Devi Reg. No: 2548007

Sreya Anil Reg. No: 2548052

July 2026

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Document Conventions	3
1.3 Scope	3
2. System Architecture	3
2.1 Architectural Overview	3
2.2 Layer Descriptions	4
2.3 External Integrations	5
3. Flow Diagrams	6
3.1 Case Reporting Flow	4
4. Algorithms and Data Structures	9
4.1 DBSCAN Hotspot Detection	9
4.2 Volunteer Reliability Score	10
4.3 Atomic Case-Claiming	10
4.4 Celery Auto-Release Countdown	10
5. Database Design	11
5.1 Normalisation Summary	11
5.2 Entity Relationship Diagram	11
5.3 Data Dictionary	12
6. Class Diagrams	20
6.1 Django Model Classes — Case and Platform	20
6.2 Django Model Classes — Animal and Adoption	21
7. Interface Design	23
8. Other Design Criteria	26
8.1 API Design	26
8.2 Security Design	27
8.3 Deployment Architecture	28

1. Introduction

1.1 Purpose

This System Design Report (SDR) provides a complete structural and behavioural design for PawMap — a community-driven mobile and web application for stray animal rescue and welfare. It covers system architecture, flow diagrams, key algorithms, database design, class diagrams, interface design, and other design criteria. This document is self-contained and does not assume familiarity with any prior project document.

1.2 Document Conventions

This document follows the IEEE Std 1016-2009 Software Design Description standard. Headings are set in Times New Roman 14pt Bold; body text is Times New Roman 12pt. Table headers use grey shading. All geographic measurements are in metres unless stated otherwise. Entity names are written in PascalCase; column names in snake_case. FK denotes Foreign Key; PK denotes Primary Key.

1.3 Scope

The design covers all five functional modules of PawMap: Stray Animal Reporting, Volunteer Dispatch and Case Management, Medical Records, Adoption and Fostering, and Analytics and Geospatial Intelligence. The technology stack is fixed to Django 4.2, React Native 0.73, React 18 + Bootstrap 5, PostgreSQL 15 + PostGIS 3.3, Celery 5.3 + Redis 7.2, scikit-learn 1.4, Google Maps Platform, and Firebase Cloud Messaging.

2. System Architecture

2.1 Architectural Overview

PawMap follows a five-layer client-server architecture. The layers are: Users, Frontend, Backend API, Data, and Data Science. All inter-layer communication is over HTTPS (TLS 1.3) using RESTful JSON conventions. Real-time case status updates use WebSocket connections via Django Channels. External services interface exclusively with the backend layer.

[System Architecture diagram]



2.2 Layer Descriptions

Layer 1 — Users

Four actor types interact with the platform: User, who reports stray animals and browses adoption listings; Volunteer, who claims rescue cases, updates case status, and submits volunteer ratings; NGO, which manages reported cases, approves volunteers, and oversees adoption requests; and Platform Admin, who reviews and approves NGO partnership requests while supervising overall platform operations.

Layer 2 — Frontend

The mobile application (React Native 0.73) serves Reporters, Volunteers, and Adopters. Primary screens include the Case Reporting Form (photo capture, GPS auto-fill, severity and aggression selectors), the Active Cases Map (real-time pins via Leaflet, filterable by species and status), the Volunteer Dashboard (claim button, status update controls, auto-release countdown), and the Adoption Listing screen. The web application (React 18 + Bootstrap 5) serves NGO Administrators and Veterinary Professionals, providing the NGO Admin Dashboard, Analytics View (DBSCAN heatmap, coverage gap layer), Report Generation, and Adopter Management Panel. Both clients communicate exclusively with the backend API over HTTPS; neither connects directly to the database.

Layer 3 — Backend API

The Django REST Framework (DRF) API server handles all business logic: JWT authentication, case lifecycle management, atomic volunteer claiming via PostgreSQL row-level locking, medical record access control, adoption queue management, and notification dispatch. Celery workers handle all asynchronous tasks: photo compression, reverse geocoding, the 30-minute auto-release countdown, DBSCAN scheduling, daily VRS computation, and monthly PDF report generation. Django Channels provides the WebSocket layer for live case status updates.

Layer 4 — Data

PostgreSQL 15 with PostGIS 3.3 stores all structured and spatial data. The key PostGIS functions used are ST_DWithin for volunteer proximity dispatch (5 km radius), ST_ClusterDBSCAN for hotspot detection, and ST_MakePoint for indexing case coordinates. Redis 7.2 serves as the Celery task broker and session cache. AWS S3 stores compressed animal photographs and generated PDF impact reports. Firebase FCM stores push notification tokens and delivers case alerts.

Layer 5 — Data Science

Scheduled Celery tasks invoke Python + scikit-learn processes. The weekly DBSCAN job clusters geo-tagged case coordinates (eps = 500 m, min_samples = 5, haversine metric) and writes hotspot polygons to PostgreSQL. The daily VRS job recomputes the Volunteer Reliability Score for every active volunteer. The spatial join job updates coverage gap flags on the Ward table. The first-of-month job generates the PDF impact report and distributes it by email.

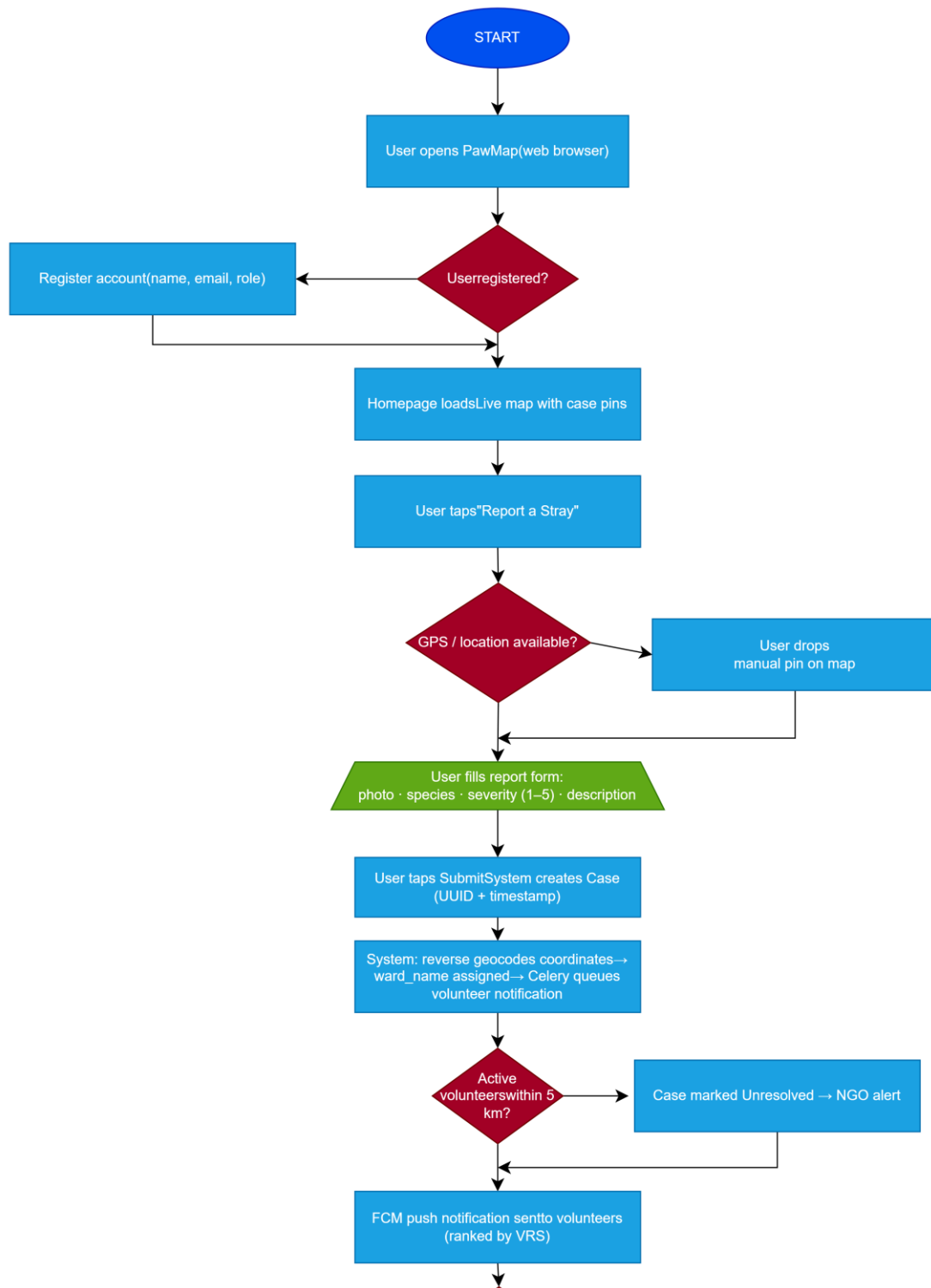
2.3 External Integrations

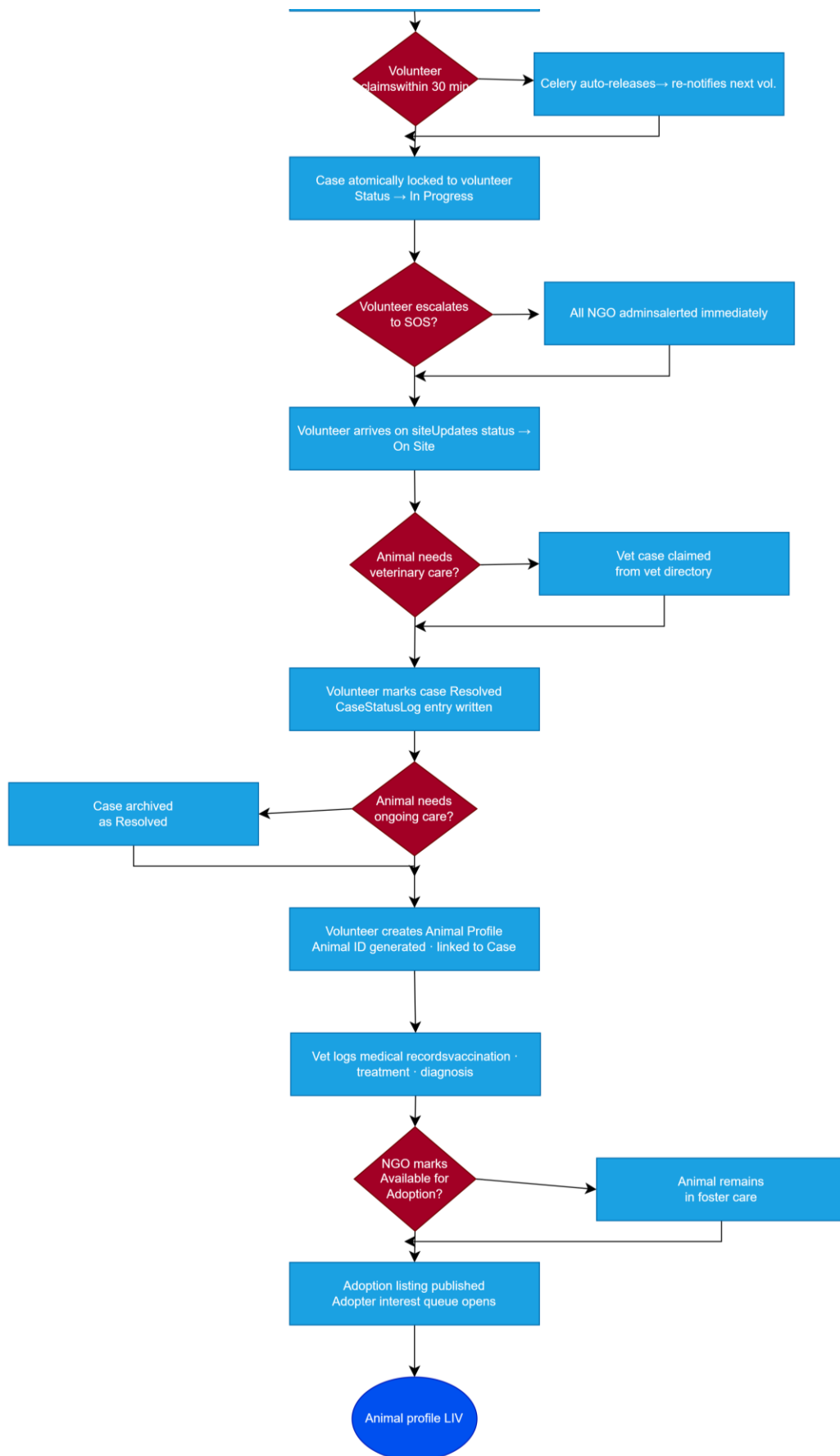
Service	Interface	Data Exchanged	Used By
Google Maps Platform	REST API (HTTPS)	Reverse geocoding (coordinates → ward_name), map tiles, geofence radius definitions	Case submission, Live Map, Coverage Gap layer
Firebase Cloud Messaging	FCM HTTP v1 API	Push notification tokens, case alert payloads, SOS broadcast messages	Volunteer dispatch, SOS escalation, vaccination reminders
AWS S3 / GCP Storage	SDK (boto3 / google-cloud-storage)	Compressed animal photographs (JPEG, max 500 KB), monthly PDF impact reports	Reporting module, Medical module, Analytics module

3. Flow Diagrams

The flow diagram below describes the core rescue lifecycle in PawMap, from the moment a community member spots a stray animal to the point the animal receives a profile and is prepared for adoption. Decision points are indicated with YES/NO branches.

3.1 Core Rescue Flow





Step	Actor	Action / Decision	Output
1	Reporter	Opens PawMap in browser, taps "Report a Stray"	App requests camera and GPS permissions
2	App	GPS available? YES → auto-fill coordinates · NO → prompt manual pin-drop on map	Coordinates populated
3	Reporter	Captures or uploads photograph; selects species, severity (1–5), aggression level, description	Form complete
4	App	Network available? YES → submit · NO → cache locally and sync on reconnect	Case queued or submitted
5	Backend	Reverse-geocodes coordinates via Google Maps API → ward_name assigned; creates Case record (UUID, UTC timestamp, PostGIS point, status = Open)	Case persisted in DB
6	Backend	Compresses photo → uploads to S3; publishes volunteer dispatch job to Celery queue	Dispatch task enqueued
7	Celery	Queries PostGIS for active volunteers within ST_DWithin(case_point, 5000 m); ranks by VRS descending	Ordered dispatch list
8	FCM	Sends push notification to ranked volunteers; case_id and confirmation returned to reporter	Reporter sees confirmation
9	Volunteer	Receives notification; opens case card; taps "Claim Case" — atomically locked via SELECT FOR UPDATE	Case status → In Progress; 30-min Celery timer starts
10	App	30 minutes elapsed with no update? YES → Celery auto-releases case, re-notifies next volunteer · NO → volunteer continues	Case reassigned or active
11	Volunteer	Volunteer escalates to SOS? YES → all NGO admins alerted immediately · NO → volunteer proceeds on site	SOS broadcast or normal flow
12	Volunteer	Arrives on site; updates status → On Site; assesses animal	Status logged in CaseStatusLog
13	Volunteer	Animal needs veterinary care? YES → vet case claimed from directory · NO → volunteer handles rescue	Vet assigned or volunteer resolves

14	Volunteer	Marks case Resolved; CaseStatusLog entry written	Case status → Resolved
15	Volunteer	Animal needs ongoing care? YES → creates Animal Profile (Animal ID generated, linked to Case) · NO → case archived	Animal profile created or case closed
16	Vet	Logs medical records against Animal ID (vaccination, treatment, diagnosis)	Medical timeline populated
17	NGO Admin	Marks animal Available for Adoption? YES → adoption listing published, adopter interest queue opens · NO → animal remains in foster care	Adoption listing live or animal in foster

4. Algorithms and Data Structures

4.1 DBSCAN Hotspot Detection

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifies statistically significant stray animal abandonment hotspots from accumulated geo-tagged case reports. It is chosen over centroid-based methods (k-means) because it does not require specifying the number of clusters in advance, handles clusters of arbitrary geographic shape, and explicitly labels low-density points as noise — preventing false hotspot identification in areas of high reporter density.

Parameter	Value	Justification
eps (neighbourhood radius)	500 m	Corresponds to approximately one Bengaluru urban ward block. Cases within 500 m are spatially proximate for NGO intervention planning.
min_samples	5	Minimum of 5 cases required to form a core point. Prevents single-incident locations from being classified as hotspots.
metric	haversine	Accounts for Earth's curvature. Required for geographic coordinate inputs (lat/lon converted to radians before input).
Input structure	numpy array (n, 2)	Each row is (latitude_radians, longitude_radians) for a resolved case.
Output	cluster labels array	Integer label per case: -1 = noise; 0, 1, 2, ... = cluster ID. Stored in HotspotCluster with polygon geometry.

Time complexity: $O(n \log n)$ with a Ball Tree spatial index (configured in scikit-learn). For 10,000 case records, clustering completes within 60 seconds on the scheduled analytics worker.

4.2 Volunteer Reliability Score (VRS)

The VRS is a weighted composite metric computed daily per volunteer. It ranks volunteers in the dispatch queue and gates SOS-tier case eligibility ($VRS \geq 0.6$ required).

Component	Weight	Formula	Data Source
Response Rate	40%	$\text{cases_responded} / \text{cases_notified}$	CaseStatusLog: notifications sent vs. claim events per volunteer
Completion Rate	40%	$\text{cases_completed} / \text{cases_responded}$	Cases with resolution_status = Resolved for that volunteer
Avg Response Time	20%	$1 - (\text{avg_response_time_min} / 30)$	Case.response_time_min; baseline = 30 min (auto-release threshold)
VRS Score	—	$0.4 \times \text{RR} + 0.4 \times \text{CR} + 0.2 \times (1 - \text{norm_RT})$	Stored in VolunteerReliabilityScore daily snapshot

Volunteers with $\text{VRS} < 0.4$ for three consecutive months have `flag_for_review` set to TRUE and an NGO Admin alert is triggered. One row per volunteer per day is written, storing all component values alongside the final score for full auditability.

4.3 Atomic Case-Claiming

To prevent two volunteers simultaneously claiming the same case, PawMap uses PostgreSQL row-level locking via SELECT FOR UPDATE inside a serialisable transaction:

```
Case.objects.select_for_update().get(pk=case_id, status='Open')
```

If a second claim arrives while the first holds the lock, the database blocks it until the first commits. The second transaction then reads `status = 'In_Progress'` and returns HTTP 409 Conflict. This guarantees exactly-one assignment without any application-layer race conditions.

4.4 Celery Auto-Release Countdown

When a volunteer claims a case, a Celery countdown task is created with a 30-minute delay. The task ID is stored on `Case.celery_task_id`. If the volunteer updates the case status before 30 minutes elapse, the countdown task is revoked. If the timer fires, the Celery worker atomically resets `Case.status` to Open, clears `Case.volunteer_id`, writes a `CaseStatusLog` entry with `is_auto_release = TRUE`, and re-triggers the dispatch flow. The underlying data structure is a Redis-backed Celery beat scheduler with task metadata stored as a Redis hash keyed by task ID.

5. Database Design

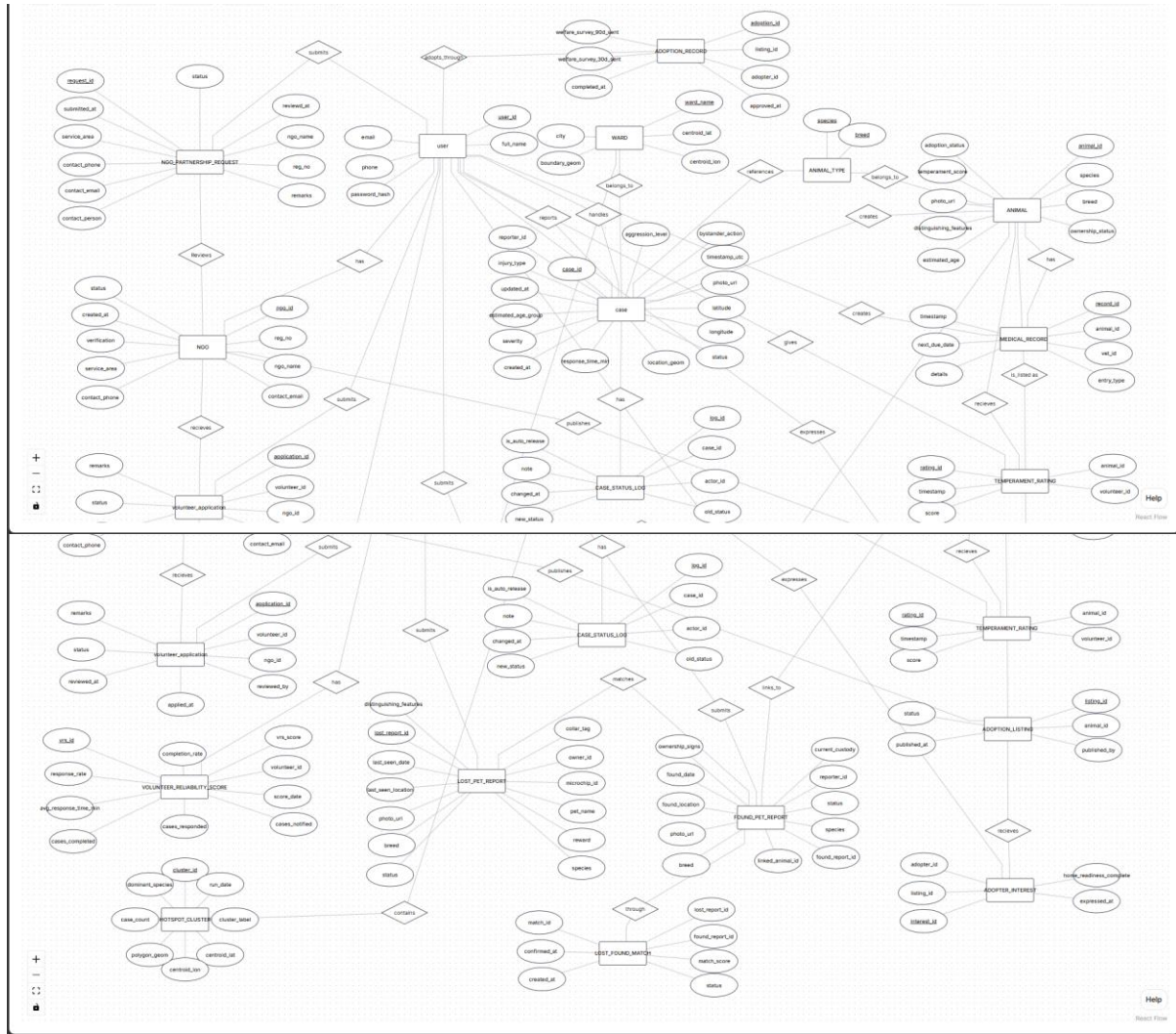
5.1 Normalisation Summary

The database schema was derived through systematic normalisation of the pilot dataset fields and the functional requirements. The following table summarises the normalisation decisions made prior to constructing the ERD.

Normal Form	Violation Found	Resolution
1NF	Medical records as flat columns (diagnosis_1, treatment_1...) create repeating groups per animal	Separated into standalone MedicalRecord table — append-only, one row per clinical entry
1NF	Multiple adopters per animal cannot be represented as flat columns	Separated into AdopterInterest table — one row per adopter per listing
2NF	Reporter attributes (age_group, prior_exp) depend only on reporter_id, not case_id	Moved to User table; pilot survey fields retained only in the CSV, not in the production schema
2NF	Ward attributes (centroid, boundary geometry) depend only on ward_name, not on any case	Separated into Ward reference table; ward_name stored on Case as a deliberate denormalisation
3NF	animal_name is functionally determined by species (e.g. only Dogs can be 'Indian pariah dog')	Created AnimalType lookup table with composite PK (species, animal_name)
3NF	vrs_score is transitively derived from response_rate, completion_rate, avg_response_time_min	All four columns retained together in VolunteerReliabilityScore for formula auditability
3NF	adoption_status is a property of the listing, not the animal itself	Moved to AdoptionListing; Animal retains only the is_adopted boolean
Deliberate denorm.	ward_name on Case is derivable from coordinates via reverse geocoding	Stored at submission time for query performance; documented as materialised field
Deliberate denorm.	Animal.temperament_score is derivable from TemperamentRating rows	Stored as materialised weighted average; recomputed on each new rating insert

5.2 Entity Relationship Diagram

The ERD below covers all 13 entities. The 13 entities are organised into two logical groups: the case-and-platform group (User, Ward, AnimalType, Case, CaseStatusLog, VolunteerReliabilityScore) and the animal-and-adoption group (Animal, TemperamentRating, MedicalRecord, AdoptionListing, AdopterInterest, AdoptionRecord, HotspotCluster). The cross-boundary FK is Case.animal_id → Animal.animal_id.



5.3 Data Dictionary

All 13 normalised tables are defined below with column names, types, constraints, and descriptions.

Table 1: User

Column	Type	Constraints	Description
user_id	VARCHAR(20)	PK	Sequential ID, format USR-0001
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(150)	UNIQUE, NOT NULL	Login credential and notification address
phone	VARCHAR(15)	UNIQUE, nullable	Contact number for NGO coordination
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash; plain text never stored
role	ENUM	NOT NULL	Reporter / Volunteer / Vet / NGO_Admin

is_verified	BOOLEAN	DEFAULT FALSE	Manual activation required for Vet and NGO_Admin
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag; deactivated accounts cannot log in
created_at	TIMESTAMP	NOT NULL	UTC account creation time
last_login	TIMESTAMP	nullable	UTC timestamp of most recent successful login

Table 2: Ward

Column	Type	Constraints	Description
ward_name	VARCHAR(100)	PK	BBMP ward name; stable government reference value
city	VARCHAR(100)	NOT NULL	Default 'Bengaluru'; supports future city expansion
centroid_lat	DECIMAL(9,6)	NOT NULL	Ward centroid latitude (WGS84)
centroid_lon	DECIMAL(9,6)	NOT NULL	Ward centroid longitude (WGS84)
boundary_geom	GEOMETRY(Polygon, 4326)	nullable	PostGIS polygon from BBMP GeoJSON; used for spatial joins in coverage gap analysis
coverage_gap_flag	BOOLEAN	DEFAULT FALSE	Set TRUE by Analytics worker when ward has unresolved cases with no volunteer response within 3 km in the past 7 days

Table 3: AnimalType

Column	Type	Constraints	Description
species	VARCHAR(50)	PK (composite part 1)	Top-level classification: Dog, Cat, Cow, Goat, Horse, Pig, Bird, Other
animal_name	VARCHAR(100)	PK (composite part 2)	Specific common name — e.g. Indian pariah dog, Tabby cat, Calf, Kid goat. Eliminates the transitive dependency animal_name → species identified in 3NF analysis.

Seed data from the pilot dataset: 8 species with approximately 30 animal_name variants. This table is populated once at deployment and extended only when new species are encountered in the field.

Table 4: Case

Column	Type	Constraints	Description
case_id	VARCHAR(20)	PK	Format CASE-0001; assigned sequentially at submission
timestamp_utc	TIMESTAMP	NOT NULL	UTC time of report submission
latitude	DECIMAL(9,6)	NOT NULL	GPS latitude of sighting (WGS84)
longitude	DECIMAL(9,6)	NOT NULL	GPS longitude of sighting (WGS84)
location_geom	GEOMETRY(Point, 4326)	NOT NULL	PostGIS Point generated from lat/lon at insert via trigger; indexed for ST_DWithin proximity queries
ward_name	VARCHAR(100)	FK → Ward	Stored denormalisation: reverse-geocoded at submission; not re-derived on each query
species	VARCHAR(50)	FK → AnimalType (composite)	Species component of the composite FK
animal_name	VARCHAR(100)	FK → AnimalType (composite)	Animal name component of the composite FK
estimated_age	ENUM	NOT NULL	Puppy / Kitten / Calf / Kid / Foal / Piglet / Juvenile / Adult / Senior
severity	SMALLINT	CHECK (1–5), NOT NULL	Reporter-assigned injury severity; 1 = healthy, 5 = critical
injury_type	VARCHAR(200)	NOT NULL	Free-text description of observed injury or condition
aggression_level	SMALLINT	CHECK (1–5), NOT NULL	Observer-assessed aggression; 1 = calm, 5 = highly aggressive; displayed prominently on the volunteer case card
prior_action_taken	ENUM	NOT NULL	none / fed_the_animal / contacted_ngo_no_response. Renamed from bystander_action; the third value clarifies that PawMap is the active rescue channel when the NGO was previously unreachable
photo_url	VARCHAR(500)	NOT NULL	S3/GCS path to compressed animal photograph (≤ 500 KB after compression)
status	ENUM	NOT NULL, DEFAULT 'Open'	Open / In_Progress / Resolved / Escalated / Unresolved
reporter_id	VARCHAR(20)	FK → User, nullable	NULL for guest submissions
volunteer_id	VARCHAR(20)	FK → User, nullable	Set on claim; CHECK constraint: NULL when status = Unresolved

animal_id	VARCHAR(20)	FK → Animal, nullable	Set post-resolution when volunteer creates the animal profile
response_time_min	DECIMAL(6,1)	nullable	Minutes between created_at and volunteer claim; primary input to VRS computation
celery_task_id	VARCHAR(255)	nullable	ID of the active auto-release countdown task; used to revoke on status update
created_at	TIMESTAMP	NOT NULL	Case creation timestamp (UTC)
updated_at	TIMESTAMP	NOT NULL	Last modification timestamp; updated on every status change

Table 5: CaseStatusLog

Column	Type	Constraints	Description
log_id	VARCHAR(20)	PK	Format LOG-0001; append-only — no UPDATE or DELETE permitted on this table
case_id	VARCHAR(20)	FK → Case, NOT NULL	The case whose status changed
actor_id	VARCHAR(20)	FK → User, nullable	User who triggered the transition; NULL for system-generated transitions
old_status	ENUM	NOT NULL	Status before the transition
new_status	ENUM	NOT NULL	Status after the transition
transition_at	TIMESTAMP	NOT NULL	UTC time of transition
note	VARCHAR(500)	nullable	Optional free-text note from volunteer or NGO Admin
is_auto_release	BOOLEAN	DEFAULT FALSE	TRUE when triggered by the Celery auto-release task, not a human actor

Table 6: VolunteerReliabilityScore

Column	Type	Constraints	Description
vrs_id	VARCHAR(20)	PK	Format VRS-0001
volunteer_id	VARCHAR(20)	FK → User, NOT NULL	Volunteer this daily score snapshot belongs to
score_date	DATE	NOT NULL	Date of computation (UNIQUE together with volunteer_id — one row per volunteer per day)

cases_notified	SMALLINT	NOT NULL	Dispatch notifications sent to this volunteer; denominator for response rate
cases_responded	SMALLINT	NOT NULL	Cases this volunteer claimed; numerator for response rate
cases_completed	SMALLINT	NOT NULL	Claimed cases marked Resolved; numerator for completion rate
avg_response_time_min	DECIMAL(6,1)	NOT NULL	Mean minutes between notification and claim for responded cases
response_rate	DECIMAL(4,3)	NOT NULL	$\text{cases_responded} / \text{cases_notified}$; stored for formula auditability
completion_rate	DECIMAL(4,3)	NOT NULL	$\text{cases_completed} / \text{cases_responded}$; stored for formula auditability
vrs_score	DECIMAL(4,3)	NOT NULL	$0.4 \times \text{response_rate} + 0.4 \times \text{completion_rate} + 0.2 \times (1 - \text{avg_response_time_min} / 30)$
flag_for_review	BOOLEAN	DEFAULT FALSE	Set TRUE when VRS < 0.4 for three consecutive months; triggers NGO Admin alert

Table 7: Animal

Column	Type	Constraints	Description
animal_id	VARCHAR(20)	PK	Format ANIM-0001; persists regardless of caregiver handover
species	VARCHAR(50)	FK → AnimalType (composite)	Species component of the composite FK
animal_name	VARCHAR(100)	FK → AnimalType (composite)	Animal name component of the composite FK
estimated_age	ENUM	NOT NULL	Puppy / Kitten / Calf / Kid / Foal / Piglet / Juvenile / Adult / Senior
distinguishing_features	TEXT	nullable	Free-text description of markings, size, or other identifying features
photo_url	VARCHAR(500)	NOT NULL	S3/GCS path to the animal's profile photograph
temperament_score	DECIMAL(3,2)	nullable	Materialised weighted average of all TemperamentRating.score values; recomputed on each new rating insert.

			More recent ratings weighted 1.5× relative to ratings older than 30 days.
is_adopted	BOOLEAN	DEFAULT FALSE	Derived flag set TRUE when the linked AdoptionRecord is completed
created_at	TIMESTAMP	NOT NULL	UTC timestamp of profile creation (set when volunteer resolves the case)

Table 8: TemperamentRating

Column	Type	Constraints	Description
rating_id	VARCHAR(20)	PK	Format RAT-0001
animal_id	VARCHAR(20)	FK → Animal, NOT NULL	The animal being rated
volunteer_id	VARCHAR(20)	FK → User, NOT NULL	The volunteer submitting the rating; only Volunteer-role users may write
score	SMALLINT	CHECK (1–5), NOT NULL	Temperament score: 1 = calm and friendly, 5 = highly aggressive
rated_at	TIMESTAMP	NOT NULL	UTC time of rating submission; used to apply the 1.5× recency weight in the aggregation formula

Table 9: MedicalRecord

Column	Type	Constraints	Description
record_id	VARCHAR(20)	PK	Format MED-0001; append-only — no UPDATE or DELETE permitted
animal_id	VARCHAR(20)	FK → Animal, NOT NULL	The animal this record belongs to
vet_id	VARCHAR(20)	FK → User, NOT NULL	The veterinary professional or NGO Admin who created the entry
entry_type	ENUM	NOT NULL	diagnosis / vaccination / treatment / weight
details	TEXT	NOT NULL	Free-text clinical details: diagnosis description, vaccine name and batch number, treatment protocol, or weight in kg
entry_at	TIMESTAMP	NOT NULL	UTC time of record creation
next_due_date	DATE	nullable	Next vaccination or follow-up due date; when within 7 days, triggers a vaccination reminder alert to the current foster or NGO Admin

Table 10: AdoptionListing

Column	Type	Constraints	Description
listing_id	VARCHAR(20)	PK	Format LST-0001
animal_id	VARCHAR(20)	FK → Animal, NOT NULL, UNIQUE	One active listing per animal at a time
published_by	VARCHAR(20)	FK → User, NOT NULL	NGO Admin who published the listing; only NGO_Admin role may publish
published_at	TIMESTAMP	NOT NULL	UTC time the listing was made public
status	ENUM	NOT NULL, DEFAULT 'Active'	Active / Pending / Completed / Withdrawn

Table 11: AdopterInterest

Column	Type	Constraints	Description
interest_id	VARCHAR(20)	PK	Format INT-0001
listing_id	VARCHAR(20)	FK → AdoptionListing, NOT NULL	The listing this interest belongs to
adopter_id	VARCHAR(20)	FK → User, NOT NULL	The registered user expressing interest; only authenticated users may submit
queue_position	INTEGER	NOT NULL	Ordered position in the adopter queue for this listing; NGO Admin may reorder
home_readiness_complete	BOOLEAN	DEFAULT FALSE	TRUE when the adopter has completed the home-readiness questionnaire; the Express Interest button is gated on this flag
expressed_at	TIMESTAMP	NOT NULL	UTC time of interest submission
UNIQUE	-	(listing_id, adopter_id)	Prevents the same adopter from expressing interest more than once per listing

Table 12: AdoptionRecord

Column	Type	Constraints	Description
adoption_id	VARCHAR(20)	PK	Format ADO-0001
listing_id	VARCHAR(20)	FK → AdoptionListing, NOT NULL, UNIQUE	One completed adoption record per listing
adopter_id	VARCHAR(20)	FK → User, NOT NULL	The approved adopter
approved_at	TIMESTAMP	NOT NULL	UTC time of NGO Admin approval
completed_at	TIMESTAMP	nullable	UTC time the adoption was marked completed; NULL while in Adoption Pending state
survey_30d_sent	BOOLEAN	DEFAULT FALSE	Set TRUE when the 30-day welfare follow-up survey email is dispatched
survey_90d_sent	BOOLEAN	DEFAULT FALSE	Set TRUE when the 90-day welfare follow-up survey email is dispatched

Table 13: HotspotCluster

Column	Type	Constraints	Description
cluster_id	VARCHAR(20)	PK	Format CLU-0001; one row per cluster per DBSCAN run
run_date	DATE	NOT NULL	Date of the DBSCAN analytics job that produced this cluster
cluster_label	INTEGER	NOT NULL	DBSCAN output label (0, 1, 2, ...); -1 noise points are never stored
centroid_lat	DECIMAL(9,6)	NOT NULL	Mean latitude of all cases in the cluster (WGS84)
centroid_lon	DECIMAL(9,6)	NOT NULL	Mean longitude of all cases in the cluster (WGS84)
polygon_geom	GEOMETRY(Polygon, 4326)	NOT NULL	Convex hull polygon of the cluster's case coordinates; rendered on the Analytics heatmap
case_count	SMALLINT	NOT NULL	Number of cases contributing to this cluster
dominant_species	VARCHAR(50)	NOT NULL	Most frequent species among cases in the cluster; used for hotspot disaggregation on the analytics map

6. Class Diagrams

The class diagrams describe Django ORM model classes, their attributes, and their methods. Each class maps directly to a PostgreSQL table. Relationships are expressed as Django ForeignKey, OneToOneField, or ManyToManyField declarations.

6.1 Django Model Classes — Case and Platform Group

Class	Key Attributes	Key Methods
User	user_id, full_name, email, phone, password_hash, role (ENUM), is_verified, is_active, created_at, last_login	is_volunteer() → bool is_ngo_admin() → bool has_sos_eligibility() → bool [checks latest VRS ≥ 0.6] deactivate() → void
Ward	ward_name (PK), city, centroid_lat, centroid_lon, boundary_geom (PostGIS Polygon), coverage_gap_flag	get_active_cases() → QuerySet[Case] flag_coverage_gap() → void clear_coverage_gap() → void
AnimalType	species (PK part 1), animal_name (PK part 2)	get_names_for_species(species: str) → list[str]
Case	case_id, timestamp_utc, latitude, longitude, location_geom, ward_name (FK), species (FK), animal_name (FK), estimated_age, severity, injury_type, aggression_level, prior_action_taken, photo_url, status, reporter_id (FK), volunteer_id (FK), animal_id (FK), response_time_min, celery_task_id, created_at, updated_at	claim(volunteer: User) → void [SELECT FOR UPDATE] release() → void [auto-release, clears volunteer_id] escalate() → void [status = Escalated, SOS broadcast] resolve(animal: Animal) → void [links animal_id, sets Resolved] get_eligible_volunteers() → QuerySet[User] [ST_DWithin 5 km, VRS DESC]
CaseStatusLog	log_id, case_id (FK), actor_id (FK nullable), old_status, new_status, transition_at, note, is_auto_release	log_transition(case, actor, old, new, note, auto) → CaseStatusLog [classmethod; append-only]
VolunteerReliabilityScore	vrs_id, volunteer_id (FK), score_date, cases_notified, cases_responded, cases_completed, avg_response_time_min, response_rate, completion_rate, vrs_score, flag_for_review	compute_for_volunteer(volunteer_id: str, date: date) → VRS [classmethod; called by Celery] get_latest(volunteer_id: str) → VRS [classmethod] check_consecutive_low(volunteer_id: str) → bool [3-month VRS < 0.4 rule]

6.2 Django Model Classes — Animal and Adoption Group

Class	Key Attributes	Key Methods
Animal	animal_id, species (FK), animal_name (FK), estimated_age, distinguishing_features, photo_url, temperament_score (materialised), is_adopted, created_at	update_temperament_score() → void [recomputes weighted avg from TemperamentRating; called on each new rating] get_medical_timeline() → QuerySet[MedicalRecord] publish_for_adoption(admin: User) → AdoptionListing mark_adopted() → void [sets is_adopted = True]
TemperamentRating	rating_id, animal_id (FK), volunteer_id (FK), score (1–5), rated_at	— (no business methods; aggregated by Animal.update_temperament_score)
MedicalRecord	record_id, animal_id (FK), vet_id (FK), entry_type (ENUM), details, entry_at, next_due_date	schedule_reminder() → void [enqueues Celery vaccination reminder if next_due_date is set] to_pdf_dict() → dict [serialises record for PDF export]
AdoptionListing	listing_id, animal_id (FK UNIQUE), published_by (FK), published_at, status (ENUM)	add_to_queue(adopter: User) → AdopterInterest approve_adopter(adopter: User) → AdoptionRecord withdraw() → void [sets status = Withdrawn]
AdopterInterest	interest_id, listing_id (FK), adopter_id (FK), queue_position, home_readiness_complete, expressed_at	— (queue ordering managed by AdoptionListing.approve_adopter)
AdoptionRecord	adoption_id, listing_id (FK UNIQUE), adopter_id (FK), approved_at, completed_at, survey_30d_sent, survey_90d_sent	complete_adoption() → void [sets completed_at, flags Animal.is_adopted] send_welfare_survey(days: int) → void [sends 30d or 90d survey email; updates survey flag]
HotspotCluster	cluster_id, run_date, cluster_label, centroid_lat, centroid_lon, polygon_geom (PostGIS Polygon), case_count, dominant_species	get_latest_clusters() → QuerySet[HotspotCluster] [classmethod; returns most recent run_date rows] to_geojson() → dict [serialises polygon for the analytics map API endpoint]

ORM relationship declarations (Django):

Relationship	Django Declaration	Notes
User → Case (reporter)	Case.reporter = ForeignKey(User, null=True, on_delete=SET_NULL, related_name='reported_cases')	Nullable for guest submissions
User → Case (volunteer)	Case.volunteer = ForeignKey(User, null=True, on_delete=SET_NULL, related_name='claimed_cases')	Nullable until claimed

Ward → Case	Case.ward = ForeignKey(Ward, on_delete=PROTECT)	PROTECT prevents ward deletion while cases exist
AnimalType → Case	Case has species + animal_name CharField pair; validated via AnimalType.get_names_for_species() in clean()	Composite FK not natively supported by Django ORM; enforced at the model validation layer
Case → Animal	Case.animal = ForeignKey(Animal, null=True, on_delete=SET_NULL)	Set post-resolution; nullable throughout the case lifecycle
Case → CaseStatusLog	CaseStatusLog.case = ForeignKey(Case, on_delete=CASCADE)	Logs cascade-deleted with the case (unlikely in production)
Animal → TemperamentRating	TemperamentRating.animal = ForeignKey(Animal, on_delete=CASCADE)	Ratings cascade-deleted with the animal profile
Animal → MedicalRecord	MedicalRecord.animal = ForeignKey(Animal, on_delete=CASCADE)	Append-only at application layer; no delete permitted via API
Animal → AdoptionListing	AdoptionListing.animal = OneToOneField(Animal, on_delete=PROTECT)	PROTECT prevents animal deletion while a listing exists
AdoptionListing → AdopterInterest	AdopterInterest.listing = ForeignKey(AdoptionListing, on_delete=CASCADE)	Interest queue cascades with the listing
AdoptionListing → AdoptionRecord	AdoptionRecord.listing = OneToOneField(AdoptionListing, on_delete=PROTECT)	One completed record per listing maximum
User → VRS	VolunteerReliabilityScore.volunteer = ForeignKey(User, on_delete=CASCADE); unique_together = ('volunteer', 'score_date')	One snapshot per volunteer per day

7. Interface Design

The table below summarises the primary screens, their inputs, and their outputs. Wireframes will be appended separately by the frontend developer.

PawMap — Get Started (Sign Up / Log In) (low-fidelity wireframe)

PawMap

[Home](#)
[Report](#)
[Volunteer](#)
[Adopt](#)
[Lost & Found](#)
[Analytics](#)
[About](#)

Get Started

Create your account

Sign up as a user, or apply as a volunteer / NGO

SIGN UP

LOG IN

I AM A...

User

Volunteer

NGO

FULL NAME

PREFERRED NGO (volunteer only)

EMAIL

PASSWORD

Volunteer / NGO signups require approval before activation and cannot claim cases or manage listings until approved.

CREATE ACCOUNT

Pending state shown after signup for Volunteer / NGO

Figure 7.1 — Get Started (Sign Up / Log In) wireframe

PawMap — Home — Easy Access panel (shown after login) (low-fidelity wireframe)

PawMap

[Home](#)
[Report](#)
[Volunteer](#)
[Adopt](#)
[Lost & Found](#)
[Analytics](#)
[About](#)

Get Started

Every stray deserves a second chance.

Report a Stray

EASY ACCESS • welcome back, [User]

Report a Stray

Adopt

Lost & Found

Medical Records

AVAILABLE FOR ADOPTION

Find your companion.

EXPRESS INTEREST

EXPRESS INTEREST

EXPRESS INTEREST

Figure 7.2 — Home screen with Easy Access panel wireframe

PawMap — Report a Stray (low-fidelity wireframe)

PawMap
Home
Report
Volunteer
Adopt
Lost & Found
Analytics
About
Get Started

STRAY ANIMAL REPORTING
Report a stray in under a minute.

TIP: TO UPLOAD PHOTO

SPECIES

Dog
Cat
Other

Specify the animal (e.g. cow, monkey, bird)...

SEVERITY (INJURY) — 2 of 5

AGGRESSION LEVEL — 1 of 5

LOCATION

Auto-detected: Koramangala 5th Block, Bengaluru

DESCRIPTION (optional)

SUBMIT REPORT

NEARBY ACTIVE CASES

ANIMAL IN IMMEDIATE DANGER?

ESCALATE TO SOS

Figure 7.3 — Stray Animal Reporting form wireframe

PawMap — Adopt — listing grid with image gallery (low-fidelity wireframe)

PawMap
Home
Report
Volunteer
Adopt
Lost & Found
Analytics
About
Get Started

ADOPTION & FOSTERING
Find your companion.

Search by name or neighbourhood...
All
Dog
Cat

Bruno

Friendly

★★★★ 4.2 / 5.0

EXPRESS INTEREST

Opens NGO-review form on click

Mitthu

Friendly

★★★★ 4.2 / 5.0

EXPRESS INTEREST

Opens NGO-review form on click

Kalu

Friendly

★★★★ 4.2 / 5.0

EXPRESS INTEREST

Opens NGO-review form on click

Figure 7.4 — Adoption listing wireframe (image gallery)

Screen	Platform	Primary Inputs	Primary Outputs	SRS Ref
--------	----------	----------------	-----------------	---------

Page 24

Case Reporting Form	Mobile	Photo capture, GPS auto-fill or pin-drop, species picker, animal_name, severity (1–5), aggression (1–5), prior_action_taken	case_id confirmation, map pin placed at submission location	REQ-01 to 05
Active Cases Map	Mobile	Filter by status, species; tap case pin	Real-time case pins with colour-coded severity; case detail card on tap	REQ-09
Volunteer Dashboard	Mobile	Claim button, status update (En Route / On Site / Resolved / Escalated), free-text note	Assigned case details, turn-by-turn link to Google Maps, auto-release countdown timer	REQ-06 to 10
Medical Record Entry	Mobile / Web	Animal profile (species, age, features, photo), diagnosis, treatment, vaccination date, next_due_date	Animal ID confirmation, appended medical timeline	REQ-11 to 15
Adoption Listing	Mobile	Express Interest button (requires verified profile)	Animal card (photo, temperament score, medical summary, foster notes), queue position	REQ-16 to 19
NGO Admin Dashboard	Web	Date range filter, ward filter, volunteer filter	Case volume KPIs, response time chart, volunteer VRS leaderboard, SOS alert feed	REQ-20 to 24
Analytics View	Web	Month selector, species filter	DBSCAN hotspot heatmap (PostGIS polygons on Leaflet map), coverage gap ward layer, trend charts	REQ-20, 21
Adopter Management Panel	Web	Approve / reorder / remove adopter from queue	Ordered adopter interest queue per animal, home-readiness status	REQ-17, 18

[Space reserved for additional wireframes: Active Cases Map, Volunteer Dashboard, Medical Record Entry, NGO Dashboard, Analytics View, and Lost & Found — to be appended separately.]

Design constraints applicable to all interfaces (from SRS Section 3.1 and Section 5.4):

- Responsive layout supporting screen widths from 375 px (mobile) to 1920 px (desktop).
- WCAG 2.1 Level AA contrast and font-size standards.
- Aggression level (1–5) and temperament score displayed prominently at the top of all case cards visible to volunteers, with colour-coded indicator: Green (1–2), Amber (3), Red (4–5).
- SOS-escalated cases trigger audio and vibration alerts on NGO Admin devices in addition to push notification (NFR-07).

8. Other Design Criteria

8.1 API Design

The PawMap REST API is implemented using Django REST Framework and documented via OpenAPI 3.0 (auto-generated by DRF Spectacular). All endpoints use JSON request/response bodies. Authentication uses JWT: 60-minute access tokens, 7-day refresh tokens (NFR-10). The following table lists the primary API endpoints.

Method	Endpoint	Auth Required	Description
POST	/api/auth/register/	None	Create new User account; role defaults to Reporter
POST	/api/auth/login/	None	Returns JWT access + refresh token pair
POST	/api/auth/refresh/	None	Refreshes access token using valid refresh token
GET	/api/cases/	Reporter+	List open cases; filterable by species, severity, ward, status
POST	/api/cases/	Reporter+	Submit new rescue case; triggers dispatch Celery job
GET	/api/cases/{id}/	Reporter+	Retrieve single case with full status history
PATCH	/api/cases/{id}/claim/	Volunteer	Atomic case claim; SELECT FOR UPDATE; returns 409 if already claimed
PATCH	/api/cases/{id}/status/	Volunteer	Update case status; cancels auto-release task on Resolved/Escalated
PATCH	/api/cases/{id}/escalate/	Volunteer	Escalate to SOS; bypasses VRS queue; broadcasts to all NGO Admins
GET	/api/animals/	Volunteer+	List animal profiles; filterable by species, adoption_status
POST	/api/animals/	Volunteer	Create animal profile post-resolution; links to case via animal_id
POST	/api/animals/{id}/medical/	Vet	Append medical record entry (diagnosis, vaccination, treatment)
POST	/api/animals/{id}/rate/	Volunteer	Submit temperament rating (1–5); triggers temperament_score recomputation
GET	/api/adoptions/	Public	List active adoption listings with animal cards
POST	/api/adoptions/{id}/interest/	Adopter	Express interest; validates home-readiness questionnaire completion

GET	/api/analytics/hotspots/	NGO_Admin	Return latest HotspotCluster polygons as GeoJSON
GET	/api/analytics/gaps/	NGO_Admin	Return wards with coverage_gap_flag = TRUE
GET	/api/analytics/vrs/	NGO_Admin	Return VRS leaderboard for all active volunteers
POST	/api/reports/generate/	NGO_Admin	Manually trigger monthly PDF impact report generation

8.2 Security Design

Security design decisions derived from SRS Section 5.3 and implemented at the architecture level:

Security Requirement	Design Decision	Implementation
TLS 1.3 in transit (NFR-09)	All client-API and API-external service communication over HTTPS port 443 only	Django SECURE_SSL_REDIRECT = True; nginx TLS termination; HSTS header enforced
AES-256 at rest (NFR-09)	Database encryption at the storage layer	AWS RDS encryption enabled (AES-256); S3 bucket server-side encryption (SSE-S3)
JWT authentication (NFR-10)	60-min access token; 7-day refresh token; mandatory re-auth after 7 days	djangorestframework-simplejwt; refresh token rotation; token blacklist on logout
Role-based access (NFR-11)	Vet and NGO_Admin require manual verification (is_verified flag) before activation	Django permission classes per endpoint; is_verified checked on login
Photo watermarking (NFR-12)	Platform logo watermark applied before photo is stored in S3	Pillow (Python) watermark applied in the Celery photo compression task
Audit log (NFR-13)	All admin actions logged; CaseStatusLog is append-only	Django signals write to CaseStatusLog; database-level INSERT-only trigger; retained 12 months
PII encryption (Constraint 2.5)	PII fields (email, phone) encrypted at the application layer	django-encrypted-model-fields; encryption key stored in AWS Secrets Manager
Volunteer GPS privacy (NFR-08)	Volunteer live location not exposed to Reporters or unverified users	Volunteer GPS stored in Redis (session only, not persisted to DB); API endpoint restricted to NGO_Admin and self

8.3 Deployment Architecture

The deployment architecture targets AWS for the development and staging environment (estimated cost \$30–50/month as per SRS Section 6.3). The following configuration is used:

Component	Service	Configuration
Backend API server	AWS EC2 t3.small	Docker container running Django + Gunicorn (4 workers); nginx reverse proxy; auto-start via systemd
Celery workers	AWS EC2 t3.small (same instance)	2 Celery worker processes sharing the EC2 instance in development; separate instance in production
Database	AWS RDS db.t3.micro	PostgreSQL 15 + PostGIS 3.3; automated daily backups; Multi-AZ disabled in development
Redis	AWS ElastiCache t3.micro	Celery broker and Django cache backend; single-node in development
Object storage	AWS S3	Bucket: pawmap-media (photos); pawmap-reports (PDFs); versioning enabled; lifecycle rule deletes old report versions after 12 months
Mobile app	Expo / React Native	OTA updates via Expo; iOS via TestFlight, Android via internal track for pilot phase
Web app	AWS S3 + CloudFront	React build deployed to S3 static site; CloudFront CDN for distribution
Container management	Docker + docker-compose	Single docker-compose.yml for development; Dockerfile per service for production